COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE31S4
DATE PERFORMED:	September 16, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	De Vera, Sophia D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK 10 IMPLEMENTING TREES

1. PROCEDURE OUTPUT

SCREENSHOTS

main-7.cpp

```cpp
#include <iostream>
#include "BinaryTree.h"

int main(){

    BinaryTree<char>* root = new BinaryTree<char>('A');
    root -> insertChildLeft('B');
    root -> insertChildRight('C');

    root ->getLeftChild() -> insertChildLeft('D');
    root ->getLeftChild() -> insertChildRight('E');
    root ->getLeftChild() -> insertChildLeft('F');

    std::cout << "The root is element: " << root -> getRootVal()<< std::endl;
    std::cout << "The left child of root 'A' is: " << root -> getLeftChild() -> getRootVal() << std::endl;
    std::cout << "The left child of 'node B' is: " << root -> getLeftChild() -> getLeftChild() -> getRootVal() << std::endl;
    std::cout << "The left child of 'node F' is: " << root->getLeftChild()->getLeftChild()->getLeftChild()->getRootVal() << std::endl;
    std::cout << "The right child of 'node B' is: " << root -> getLeftChild() -> getRightChild() -> getRootVal() << std::endl;
    std::cout << "The right child of 'root A' is: " << root -> getRightChild() -> getRootVal() << std::endl;
    std::cout << "The right child of 'node C' is: " << root -> getRightChild() -> getRightChild() -> getRootVal() << std::endl;

    root -> getRightChild() -> setRootVal('S');
    std::cout << "The NEW Right Child of 'root A' is: " << root->getRightChild()->getRootVal() << std::endl;
    std::cout << "The right child of 'node S' is: " << root->getRightChild()->getRightChild()->getRootVal() << std::endl;

    return 0;
}
```

BinaryTree.h

```cpp
#ifndef BINARYTREE_H
#define BINARYTREE_H

template <typename T>
class BinaryTree{
    private:
        T key;
        //pointers for the left and right child
        BinaryTree<T>* leftChild;
        BinaryTree<T>* rightChild;
    public:
        //  Constructor
        BinaryTree(T rootObj);
        // insert a child on the left node:
        void insertChildLeft(T newNode);
        void insertChildRight(T newNode);

        BinaryTree<T>* getLeftChild();
        BinaryTree<T>* getRightChild();

        T getRootVal();
        void setRootVal(T newNode);
};
```

BinaryTree.h

```cpp
// Constructor
template<typename T>
BinaryTree<T>::BinaryTree(T rootObj){
    key = rootObj;
    leftChild = nullptr;
    rightChild = nullptr;
}

// insert leftchild:
template<typename T>
void BinaryTree<T>::insertChildLeft(T newNode){
    if(leftChild == nullptr){
        leftChild = new BinaryTree<T>(newNode);
    } else {
        BinaryTree<T>* newTreeNode = new BinaryTree<T>(newNode);
        newTreeNode->leftChild = leftChild;
        leftChild = newTreeNode;
    }
}
```

```cpp
// insert rightchild
template<typename T>
void BinaryTree<T>::insertChildRight(T newNode){
    if(rightChild == nullptr){
        rightChild = new BinaryTree<T>(newNode);
    } else {
        BinaryTree<T>* newTreeNode = new BinaryTree<T>(newNode);
        newTreeNode->rightChild = rightChild;
        rightChild = newTreeNode;
    }
}

template<typename T>
BinaryTree<T> *BinaryTree<T>::getLeftChild(){
    return leftChild;
}

template<typename T>
BinaryTree<T> *BinaryTree<T>::getRightChild(){
    return rightChild;
}

template<typename T>
T BinaryTree<T>::getRootVal(){
    return key;
}
```

```cpp
template<typename T>
void BinaryTree<T>::setRootVal(T newNode){
    key = newNode;
}

#endif
```

```
PS C:\DSA\SW10> & C:\DSA\SW10\main-7.exe
The root is element: A
The left child of root 'A' is: B
The left child of 'node B' is: F
The left child of 'node F' is: D
The right child of 'node B' is: E
The right child of 'root A' is: C
```

OBSERVATION ANALYSIS	- **In this seatwork, we learned the fundamentals of a tree diagram where each branch is called a child. We also learned how a binary tree uses nodes and pointers (like the linked lists) to connect the root to its left and right children. We used insertChildLeft() and insertChildRight() as functions to specify where to insert new elements.** - **I also learned that when a child already exists, the insertion function can move the existing child below the newly inserted node.** - **Overall, binary tree structure is also one sorting technique that I think will be helpful when it comes to connecting data that requires classification, just like the example from the PPT where the Animalia Kingdom was presented.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.